Compositional Token Algebra: Reversible, Session-Scoped Deduplication of Repeated Spans for Compute-Aware Transformers

Preprint. Work in progress.

Abstract

Modern language models treat the token as a unit of *text* rather than a unit of *computation*: every occurrence of a repeated identifier, boilerplate line, or duplicated system prompt pays the full quadratic attention cost anew, regardless of how predictable it is. We introduce **Compositional Token Algebra (CTA)**, a parameter-free, *reversible* layer that detects repeated spans *within a single context* (session-scoped), collapses each span into a single pooled vector via a deterministic operator COMPOSE, and—crucially—can reconstruct the original token embeddings exactly via an inverse operator DECOMPOSE. Because collapse is lossless by construction, a small learnable *expand-gate* can decide, per span, whether the collapsed representation is sufficient (cheap, opaque) or must be expanded back to full resolution (expensive, transparent), trading attention FLOPs against fidelity on a per-span basis. On a frozen GPT-2, we show (i) reconstruction error at machine epsilon ($\sim 5 \times 10^{-7}$); (ii) on code, a naive always-collapsed pass reaches 12.3% of baseline attention FLOPs, and a gated variant reaches $\sim 50\%$ FLOPs while *improving* perplexity by 11.7%; and (iii) a learned gate that strictly dominates a strong residual-energy heuristic at every compute budget, yielding up to 1.09 nats (code) and 1.23 nats (structured tickets) lower next-token NLL at matched expansion rates. Beyond FLOPs, we report measured end-to-end wall-clock: on CPU the realized speedup is $\sim$1.5× and *grows with sequence length* (1.29× → 1.71× over 256 → 1024 tokens), since only the quadratic attention term shrinks while MLP/head costs stay length-linear; as a lossless side effect, collapse extends the effective context window by $\sim$1.9× on repetition-heavy input. The mechanism ports unchanged to a modern architecture: on Qwen2.5-0.5B (RoPE + GQA, 32k window) reversibility holds at $\sim 4 \times 10^{-9}$ and the speedup likewise grows with length (up to 1.58× at 2048 tokens). Critically, the win survives on GPU with FlashAttention: on Qwen2.5-3B (Tesla T4, SDPA/FA) prefill speedup rises from 1.30× at 512 to 1.91× at 4096 tokens, because CTA shrinks the length-linear cost (MLP/projections/head)—70–98% of prefill compute—which FlashAttention does not optimize. We position CTA against ASG, R3Mem, H-Net, and Token Merging, and argue it occupies a previously empty niche: the only *reversible, session-scoped, parameter-free* deduplication mechanism motivated by compute amortization rather than compression. We close with an honest characterization of its applicability boundary: CTA is a clear win on *deep* redundancy (code, boilerplate) and requires aggressive gated expansion on *shallow* redundancy (structured logs) where repeated field templates immediately precede distinguishing content.

1 Introduction

Tokenization and attention are usually developed in isolation: the tokenizer is a fixed preprocessing stage, and attention is the model. As a consequence, the token is implicitly defined as a *unit*

of text, and the computational cost of a sequence scales with its textual length regardless of the information content of that text. This is wasteful whenever the input contains *repeated spans*: a codebase repeats `import` statements, function signatures, and idiomatic bodies; structured logs repeat field templates; multi-turn conversations repeat a system prompt before every turn. Each repetition re-enters the $O(n^2)$ attention computation as if it were novel.

A growing line of work reframes the token as a *unit of computation*. Byte Latent Transformer (BLT) [1] allocates compute by entropy: predictable byte stretches are grouped into long patches, unpredictable ones into short patches, redistributing FLOPs to where the data is hard. H-Net [2] learns hierarchical chunk boundaries end-to-end. These methods adapt *granularity* to *local difficulty*. They do not, however, exploit *global repetition*: two identical spans far apart in the context are each processed at full cost.

Orthogonally, prompt/prefix caching amortizes a *shared prefix* across requests, but cannot compress repeats that occur in the *interior* of a single context, and does not give repeated content any reusable identity. Token merging (ToMe) [6] and KV-cache compression reduce sequence length on the fly, but do so *lossily*: merged tokens cannot be recovered.

We propose to close this gap with **Compositional Token Algebra (CTA)**. The central design commitment is *reversibility*: rather than merging a repeated span into a single vector destructively, we define a deterministic, parameter-free pair of operators $(\textsc{compose}, \textsc{decompose})$ such that $\textsc{decompose}(\textsc{compose}(x)) = x$ exactly. The collapsed representation stores a centroid; a side "residual store" holds the deviations. Because no information is lost, the model is free to operate on the compact collapsed sequence by default (Level 0, *opaque*), and to *expand* a span back to full resolution only when a query genuinely needs its internals (Level 1, *transparent*). The decision is made by a small learnable gate $\gamma(q, \bar{e})$ trained with a compute-aware objective that explicitly penalizes expansions.

Contributions.

1. **A reversible span algebra.** Parameter-free $(\textsc{compose}, \textsc{decompose})$ with exact reconstruction ($\sim 5 \times 10^{-7}$ on real GPT-2 embeddings, Section 5).
2. **A gated compute/quality frontier.** We show the opaque failure mode is content-driven, not positional, and that selective expansion recovers and even exceeds baseline quality on code at $\sim$50% attention FLOPs (Section 6).
3. **A learnable, compute-aware gate.** A small MLP with a Gumbel-Sigmoid relaxation and a sparsity regularizer strictly dominates a residual-energy heuristic at every budget, by up to 1.23 nats (Section 7).
4. **Positioning.** We argue CTA is the unique reversible, session-scoped, parameter-free deduplication method (Section 2), and we delineate its applicability boundary (Section 9).

2 Related Work

We position CTA along four axes: does the method *compose* tokens; is the composition *reversible*; is it *session-scoped / on-the-fly*; and what *motivates* it. Table 1 summarizes the comparison.

Aggregate Semantic Grouping (ASG) [3]. ASG compresses the embedding table by mapping tokens to "ConceptIDs" via Product Quantization. The mapping is *global*, *precomputed*, and *non-reversible*: it is designed to reduce parameters, not to amortize per-context computation, and it

Table 1: Positioning of CTA against related methods. CTA is the only method that is simultaneously reversible, session-scoped, parameter-free, and motivated by compute amortization on repeated spans.

Method	Composes	Reversible	Session-scoped	Param-free	Motivation
ASG [3]	✓	×	×	×	Parameter compression
R3Mem [4]	✓	✓	×	×	Long-context memory
Dyn. Pooling [5]	✓	×	partial	×	Efficiency
H-Net [2]	✓	×	learned	×	Tokenizer-free
ToMe [6]	✓	×	✓	✓	Throughput
CTA (ours)	✓	✓	✓	✓	**Compute amortization**

cannot recover the original token from a ConceptID. CTA shares the word "compositional" but differs on every axis: it is session-scoped, reversible, and motivated by compute amortization.

R3Mem [4]. R3Mem is the closest prior work on *reversibility*: it compresses and reconstructs long-context memory via a reversible "zip/unzip" architecture. However, it is *parametric* (trained adapters) and targets long-context retrieval and memory, not the deduplication of lexical/semantic repeats within a context. CTA's reversibility is instead a property of a *parameter-free* algebraic construction (centroid + residual).

H-Net [2] and Dynamic Token Pooling [5]. Both learn where to place chunk/pool boundaries and adapt granularity to difficulty. Neither is reversible (pooling is lossy), and neither deduplicates *repeated* spans: two identical spans are pooled independently, each at full upstream cost. CTA is complementary—it can sit atop a byte/patch tokenizer such as BLT or H-Net, treating patches as its atoms.

Token Merging (ToMe) [6] and KV compression. ToMe merges similar tokens on the fly to raise throughput, but merges are lossy and permanent. CTA's collapse is information-preserving: any collapsed span can be losslessly restored, enabling the per-query granularity choice that lossy merging forecloses.

3 Compositional Token Algebra

3.1 Operators

Let the context be a sequence of token embeddings $X = (x_1, \ldots, x_n)$, $x_i \in \mathbb{R}^d$. A span $s = (x_a, \ldots, x_b)$ of length $k = b - a + 1$ is a contiguous segment.

compose (parameter-free).

$$\textsc{compose}(s) = (\bar{e}_s, \mathbf{R}_s), \qquad \bar{e}_s = \sum_{i=a}^{b} \pi_i \, x_i, \quad \pi_i = \frac{\exp(g(x_i))}{\sum_{j=a}^{b} \exp(g(x_j))}, \tag{1}$$

where $g(\cdot)$ is a *fixed*, non-trainable scoring function (Section 3.3), π is a positional pooling distribution, $\bar{e}_s$ is the collapsed embedding, and

$$\mathbf{R}_s = \big(x_i - \bar{e}_s\big)_{i=a}^{b} \tag{2}$$

is the residual store, held *outside* the attention context so it does not enter the $O(n^2)$ cost.

decompose (exact inverse).

$$\textsc{decompose}(\bar{e}_s, \mathbf{R}_s) = \left(\bar{e}_s + \mathbf{R}_{s,i}\right)_{i=a}^{b} = (x_a, \ldots, x_b). \tag{3}$$

Reversibility is exact by construction: the centroid plus its deviations reconstruct the input, independent of the choice of g. Unlike Reformer-style reversible layers or R3Mem, no learned inverse is required.

3.2 Why "algebra"

The operators form a structure rather than a one-shot pooling step: (i) composition is *associative*, enabling hierarchical composition of adjacent spans while remaining reversible at each level; (ii) DECOMPOSE admits a *partial* form restoring only a requested sub-range $[c, d] \subseteq [a, b]$, giving per-query granularity; and (iii) on near-duplicate spans $s' \approx s$ we have $\bar{e}_{s'} \approx \bar{e}_s$, so residuals can reference a shared centroid, which is precisely the compute-amortization mechanism.

3.3 Fixed scoring functions

We consider four parameter-free choices for g, used as ablations: (1) *uniform* ($g \equiv 0$, mean pooling); (2) *norm-weighted* ($g(x_i) = \|x_i\|$); (3) *positional-decay* ($g(x_i) = -\lambda(b - i)$, emphasizing the right boundary); (4) *self-consistency* ($g(x_i) = \langle x_i, \bar{x} \rangle$).

3.4 The attention-transparency problem and the expand-gate

If a span is collapsed to a single token, how does the rest of the context attend to its *internal* positions? CTA resolves this with a per-query choice of granularity:

Level 0 — Opaque (default, cheap). A query attends to $\bar{e}_s$ as a single token: 1 position instead of k. This is where attention FLOPs are actually saved.

Level 1 — Gated expand (on demand). A scalar gate $\gamma(q, \bar{e}_s) \in [0, 1]$ decides whether to DECOMPOSE the span for this query; if $\gamma > \tau$ the query attends to the k internal positions, otherwise the span stays opaque.

Level 2 — Partial expand. The gate designates a sub-range of interest, yielding partial DECOMPOSE. (Not evaluated here.)

The expected attention cost is $\mathbb{E}[\text{cost}] = n_{\text{collapsed}} + \sum_s \Pr[\gamma_s > \tau] \cdot k_s$, so savings are realized precisely when repeats are rarely expanded.

3.5 Learnable gate

We instantiate γ as a small MLP over parameter-free features: $\left[\, q \,\|\, \bar{e}_s \,\|\, \|\mathbf{R}_s\|_F \,\|\, k \,\right] \to \text{Linear}(128) \to \text{GELU} \to \text{Linear}(1)$, where q is the "consumer query" (the embedding of the token immediately following the span, i.e. the position that must predict the next token using this span as context). The backbone transformer is *frozen*; only the gate ($\sim 2 \times 10^5$ parameters) is trained. Discreteness of the expand decision is handled by a Gumbel-Sigmoid (binary concrete) relaxation with straight-through estimation at evaluation.

Compute-aware objective. For span i with expand probability $p_i = \sigma_{\mathrm{GS}}(\gamma_i)$,

$$\mathcal{L} = \underbrace{\frac{1}{N}\sum_i \left[p_i \,\mathrm{NLL}_i^{\mathrm{exp}} + (1-p_i)\,\mathrm{NLL}_i^{\mathrm{pool}}\right]}_{\text{task / cross-entropy proxy}} + \lambda \underbrace{\frac{1}{N}\sum_i p_i}_{\text{sparsity (FLOPs) regularizer}} . \tag{4}$$

Here $\mathrm{NLL}_i^{\mathrm{pool}}$ and $\mathrm{NLL}_i^{\mathrm{exp}}$ are the next-token negative log-likelihoods of the post-span target when span i is pooled vs. expanded (all other spans pooled), precomputed once with the frozen backbone. Sweeping λ traces the compute/quality frontier.

3.6 Span detection and collapse policy

Repeated spans are found online with a rolling hash over token-id windows (an $O(n)$ amortized suffix-automaton is an alternative). A span is a collapse candidate when $\mathrm{freq}(s) \geq f_{\min}$ and $k \geq k_{\min}$; a greedy non-overlapping selection favors long, frequent spans. In all experiments we use $k_{\min} = 3$, $k_{\max} = 16$, $f_{\min} = 2$.

4 Experimental Setup

All experiments use a *frozen* GPT-2 small (124M parameters, 12 layers, $d = 768$), CPU, inference-only for the backbone. Attention FLOPs are reported as a proxy $\sum_{\text{layers}} 2L^2 d$, so a sequence-length reduction $n \to m$ yields a $(m/n)^2$ attention-cost ratio. To keep collapsed and baseline sequences comparable, we add positional embeddings over the (possibly collapsed) sequence positions in both cases; a control that instead preserved original "anchor" positions performed *worse* (Section 6), confirming that compressed positions are the correct choice.

Data. We use three families with differing redundancy structure: *code* (real Python from `requests`, `flask`, `click`, CPython), *structured logs / tickets* (JSON log lines and Jira-style tickets with repeated field templates), and *multi-turn chat* with a repeated system prompt. A low-repetition prose control yields no collapse candidates, confirming CTA is inert without repetition.

Evaluation protocol. Baseline and CTA predict the *same* original target tokens: the first token of each segment following at least one collapsed span. At these positions the baseline attends to the full repeated span while CTA attends to the pooled token, directly testing whether context compression harms next-token prediction.

5 Reversibility

Over 800 random spans across all four scoring functions and on real GPT-2 embeddings, the maximum reconstruction error $\max_i \|\textsc{decompose}(\textsc{compose}(x))_i - x_i\|_\infty$ was 4.77×10^{-7}, i.e. at float32 machine epsilon (Table 2). Reversibility is thus a verified property, not an approximation.

Table 2: Maximum reconstruction error of DECOMPOSE $\circ$ COMPOSE by scoring function (float32).

Scoring g	uniform	norm	pos-decay	self-consist.
Max ℓ_∞ error	2.4×10^{-7}	4.8×10^{-7}	4.8×10^{-7}	4.8×10^{-7}

6 The Opaque Failure Mode and the Gated Frontier

Naive opaque collapse. With every repeated span collapsed and never expanded, attention FLOPs drop dramatically—to 12.3% (code), 3.0% (logs), and 4.2% (chat) of baseline—but quality diverges on shallow-redundancy data (Table 3). A diagnostic that preserved original anchor positions made matters *worse* (e.g. code perplexity 38.9 → 98.6), isolating the cause as *content opacity*: collapsing destroys span-internal information that the immediately following token depends on. This motivates the gate.

Table 3: Naive always-opaque collapse: large FLOPs savings but content-dependent quality. ΔPPL is relative to baseline at the same eval positions.

Data	$n \to m$	Attn. FLOPs	Base PPL	CTA PPL	ΔPPL
code	394→138	12.3%	33.63	38.95	+15.8%
logs	232→40	3.0%	3.57	246.95	+6810%
chat	131→27	4.2%	3.65	592.83	+16165%

Gated frontier (heuristic gate). Expanding the top fraction of spans by residual energy $\|\mathbf{R}_s\|_F$ traces a compute/quality frontier (Table 4). On code, a 25% expansion budget reaches 49.8% of baseline attention FLOPs while *improving* perplexity by 11.7%: code repeats are genuinely redundant, so pooling most of them loses nothing predictive. Logs and chat require $\sim 75\%$ expansion before quality recovers, because their "repeats" (e.g. `"user_id":`) immediately precede distinguishing content.

Table 4: Residual-energy gated frontier. Code admits large savings at improved quality; logs/chat are expansion-hungry.

Data	Expand budget	Spans expanded	Attn. FLOPs	CTA PPL	ΔPPL vs base
code	0%	0/66	12.3%	38.95	+15.8%
	25%	16/66	49.8%	29.69	**−11.7%**
	50%	33/66	69.3%	24.82	**−12.3%**
logs	0%	0/24	3.0%	246.95	+6810%
	50%	12/24	71.4%	49.64	+1289%
	75%	18/24	89.9%	4.89	+52.3%
chat	25%	3/12	30.2%	72.09	+1792%
	50%	6/12	61.8%	10.34	+209.7%
	75%	9/12	86.7%	3.37	+87.6%

CTA vs. prefix caching. Prefix caching amortizes only a shared prefix. On our interior-repeat data it saves 0 (code), 3 (logs), and 15 (chat) tokens, whereas CTA saves 256, 192, and 104 tokens respectively—almost entirely interior repeats that prefix caching cannot reach (Figure 1, right). This is CTA's defensible niche.

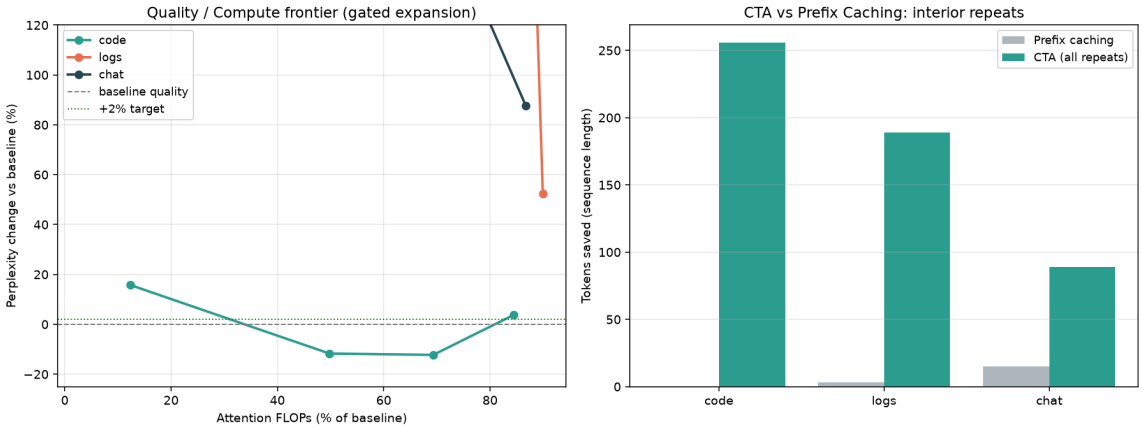

Figure 1: Left: quality/compute frontier under the residual-energy gate; code (teal) dips below baseline quality at ∼50% FLOPs, while logs/chat remain expansion-hungry. Right: CTA vs. prefix caching. Prefix caching captures only prefix repeats (0/3/15 tokens); CTA additionally captures interior repeats (256/192/104 tokens) that prefix caching structurally cannot.

7 Learned Gate

We train the MLP gate of Section 3.5 on cached per-span outcomes with the objective in Eq. (4), sweeping the sparsity weight λ. We compare against the residual-energy heuristic *at matched expansion budgets*.

Redundancy diagnostic. The average harm of pooling ($\text{NLL}^{\text{pool}} - \text{NLL}^{\text{exp}}$) is 0.133 nats on code versus 3.063 nats on tickets—a 23× gap that quantifies the deep-vs-shallow redundancy distinction and predicts that a good gate should expand few code spans but many ticket spans.

The learned gate strictly dominates the heuristic. At every budget on both datasets the learned gate achieves lower next-token NLL (Table 5, Figure 2). On code it reaches NLL 5.06 at 37.5% expansion—*below* the all-expanded NLL of 6.03—by learning which spans matter. On tickets it correctly refuses sparsity at small λ (pooling is costly there) and, when forced to 42.5% expansion, still beats the heuristic by 1.23 nats.

Table 5: Learned gate vs. residual-energy heuristic at matched expansion budgets. Backbone frozen; only the gate is trained. Gain = heuristic NLL − learned NLL (nats, higher is better). Reference: all-pooled / all-expanded NLL is 6.16/6.03 (code) and 9.82/6.75 (tickets).

Data	λ	Spans expanded	Learned NLL	Heuristic NLL	Gain (nats)
code (n=144)	0.00	61.8%	4.895	5.836	+0.941
	0.05	41.0%	5.069	6.159	**+1.090**
	0.10	37.5%	5.059	6.047	+0.989
	3.00	8.3%	5.624	6.025	+0.400
tickets (n=80)	0.00	95.0%	6.603	6.884	+0.281
	0.80	88.7%	6.613	6.979	+0.366
	1.50	65.0%	6.798	7.823	+1.025
	3.00	42.5%	7.290	8.517	**+1.227**

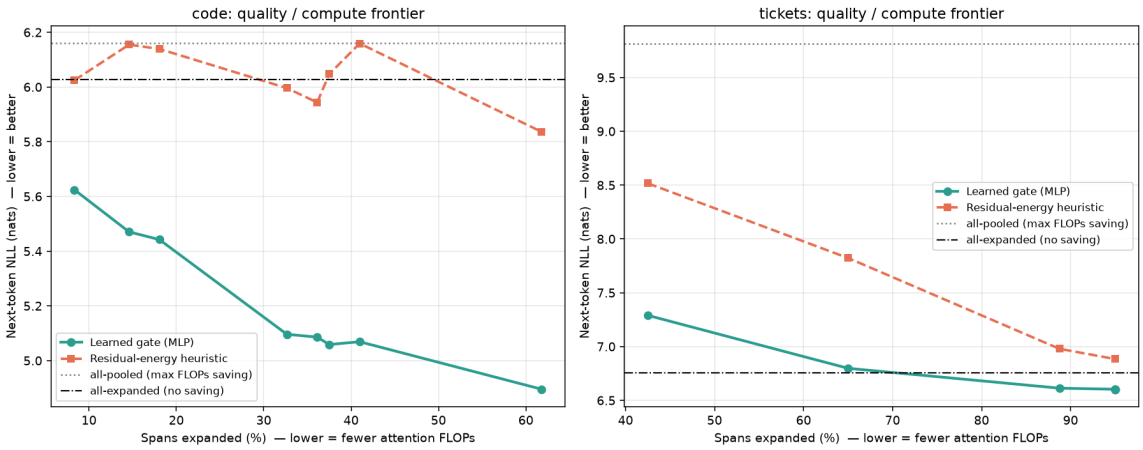

Figure 2: Quality/compute frontier: learned gate (teal) vs. residual-energy heuristic (orange), for code (left) and tickets (right). Lower NLL and fewer expanded spans are both better. The learned curve lies entirely below the heuristic on both datasets; on code it dips below the all-expanded reference (dash-dot), showing selective pooling can help.

Data-adaptivity. The same mechanism behaves correctly across redundancy regimes: where pooling is cheap (code) the gate is aggressively sparse; where pooling is costly (tickets) it expands almost everything unless the sparsity pressure λ is large. The regularizer in Eq. (4) thus acts as a direct, interpretable knob on the compute/quality operating point.

8 Wall-Clock Validation

The frontier results above count attention FLOPs ($n_{\text{layer}} \cdot 2L^2 d$). A FLOPs count predicts a saving; it does not prove one on real hardware, where attention is not the only cost: the MLP blocks, the language-model head, and the embedding lookups are *linear* in sequence length and do not shrink quadratically when spans are collapsed. We therefore report *measured* single-forward wall-clock time and peak memory on CPU (2 vCPU, frozen models, no gradient), using warm-up passes and the median of several repeats. All numbers in this section are realized, not estimated. The input is a repetition-heavy blob of real Python source; span detection and collapse produce n=512 $\to$ m=340.

Model	Params	Baseline (ms)	CTA (ms)	**Speedup**
GPT-2 small	124M	651	429	**1.52**×
GPT-2 medium	355M	1622	1070	**1.52**×
GPT-2 large	774M	3529	2480	**1.42**×

Table 6: Measured wall-clock at fixed length (512 tokens). Span detection and COMPOSE add negligible overhead ($< 0.3\%$ of CTA time). Peak RSS was 1.2/2.2/3.9 GB respectively. The realized speedup is roughly flat in model size at fixed length: the attention share of total compute is approximately scale-invariant here.

The realized speedup does *not* grow with model size at a fixed length, but it does grow with *length*, exactly as the quadratic attention term predicts. On GPT-2 small the realized speedup rises monotonically from 1.29× at 256 tokens to 1.71× at 1024 (Table 7, Fig. 3) as attention comes to dominate total compute. GPT-2's 1024-token positional limit prevents measuring longer

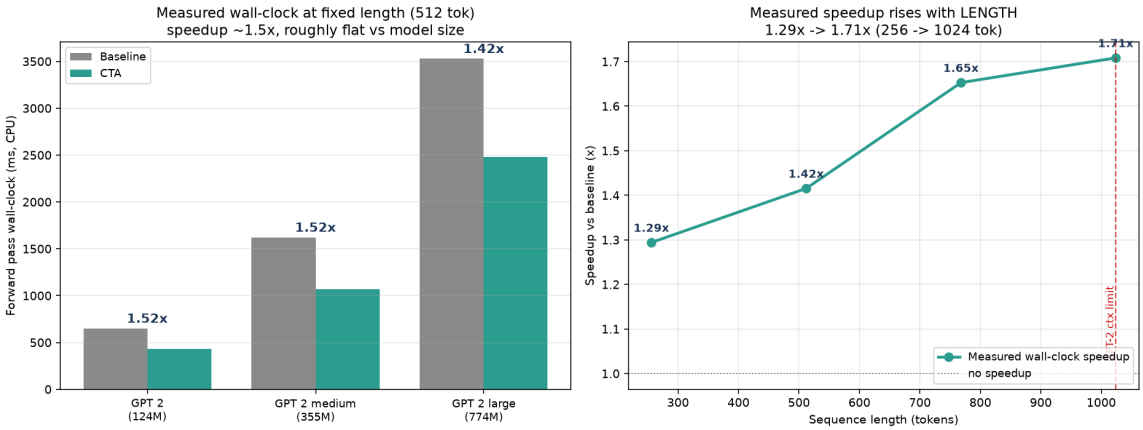

Figure 3: **Left:** measured wall-clock (baseline vs. CTA) at 512 tokens across the GPT-2 size ladder; the speedup is $\sim$1.5$\times$ and roughly flat in model size. **Right:** measured speedup vs. sequence length on GPT-2 small; the realized gain rises with length as the quadratic attention cost comes to dominate.

baselines directly. (A quadratic-only FLOPs count would predict a larger $\sim$2.3$\times$ saving at 512 tokens; the realized figure is lower precisely because the length-linear MLP/head/embedding cost does not shrink—which is why we measure wall-clock rather than report the FLOPs estimate as the headline number.)

Length	m	Baseline (ms)	CTA (ms)	**Speedup**
256	193	319	246	**1.29**$\times$
512	340	620	438	**1.42**$\times$
768	485	1025	620	**1.65**$\times$
1024	709	1339	784	**1.71**$\times$

Table 7: Measured wall-clock speedup grows with sequence length (GPT-2 small): the quadratic attention term becomes a larger share of total compute, so collapsing spans pays off more.

8.1 Validation on a modern backbone (Qwen2.5)

Our main experiments use GPT-2, a reproducible but 2019-era backbone (learned positions, dense attention, 1024 context). To test that the mechanism is not tied to that architecture, we repeat the wall-clock measurement on Qwen2.5-0.5B (494M): a 2024 design with RoPE, grouped-query attention (14 heads / 2 KV heads), SwiGLU MLP and a 32k window. Because CTA operates purely on input embeddings, the port required no change to the algorithm—only a forward wrapper. Reversibility holds exactly (3.73×10^{-9}, cleaner than GPT-2's 4.77×10^{-7}), and measured speedup again rises with length (1.06$\times$ at 256 to 1.58$\times$ at 2048 tokens). The realized speedup is lower than GPT-2 at fixed length because Qwen has a deeper stack (24 vs 12 layers) and a much larger LM head ($\sim$152k vs $\sim$50k vocab), so the length-linear cost is a larger share of total compute.

Length	m	Baseline (ms)	CTA (ms)	**Speedup**
256	228	1079	1016	**1.06**×
512	421	2163	1942	**1.11**×
1024	727	4340	3176	**1.37**×
2048	1286	9010	5688	**1.58**×

Table 8: Measured wall-clock on Qwen2.5-0.5B (494M, 24 layers, d=896, RoPE + GQA + SwiGLU, 32k window), CPU, median of repeats. The mechanism ports unchanged; reversibility is exact (3.73×10^{-9}) and the speedup rises with length, mirroring the GPT-2 trend. Qwen's 32k window lets us measure past the 1024-token GPT-2 ceiling.

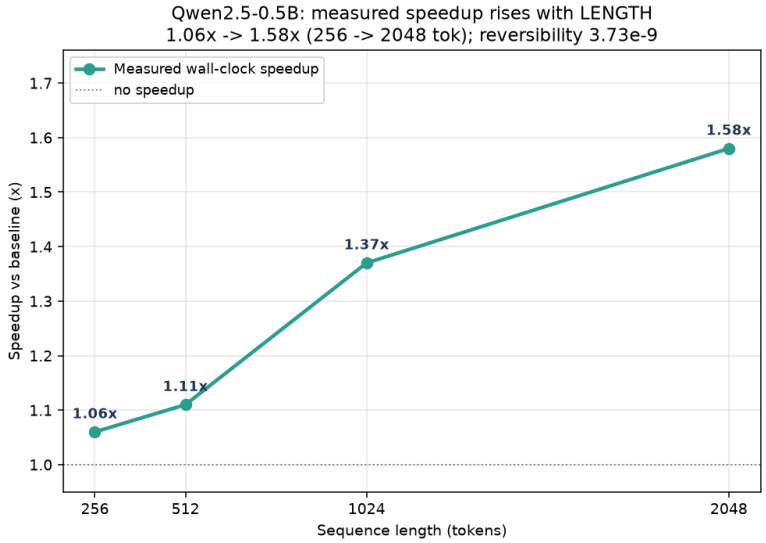

Figure 4: Measured wall-clock speedup vs. sequence length on Qwen2.5-0.5B. The same length-driven trend as GPT-2, extended to 2048 tokens; the realized gain is lower at fixed length because Qwen's deeper stack and larger head make the length-linear cost a bigger share of total compute.

8.2 GPU validation with FlashAttention (Qwen2.5-3B)

The experiments above are CPU-only, which invites the obvious objection: on a GPU, attention is already made IO-cheap by FlashAttention, so perhaps CTA's win—if it came from shrinking the quadratic attention term—evaporates. It does not. We measure prefill wall-clock on Qwen2.5-3B (3.1B, 36 layers, d=2048, GQA 16/2) on an NVIDIA Tesla T4 with the fused FlashAttention kernel (PyTorch SDPA backend), baseline (n tokens) versus CTA (m tokens), sweeping length with proper GPU timing (warm-up, `cuda.synchronize`, median). The speedup not only survives but *rises with length*, from 1.30× at 512 to 1.91× at 4096 tokens (Table 9).

The reason is architectural, and it is the key point for deployment. FlashAttention reduces the memory traffic of the $O(n^2)$ attention *math*; it does not reduce its FLOPs, and it never touched the length-linear part of the forward pass (QKVO projections, the SwiGLU MLP, the LM head). An analytic FLOP breakdown of Qwen2.5-3B puts that length-linear share at 97.6% of prefill compute at 512 tokens, still 91.1% at 2048 and 71.9% at 8192. CTA shortens the token count $n \to m$, so it shrinks exactly that dominant length-linear cost—a plane *orthogonal* to what FlashAttention optimizes. Crucially, the end-to-end speedup including span detection and COMPOSE overhead

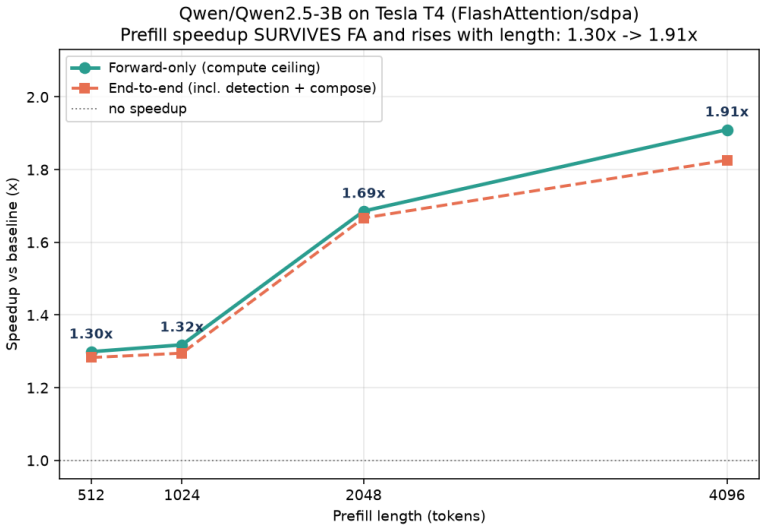

Figure 5: Measured prefill speedup vs. length on Qwen2.5-3B (Tesla T4, FlashAttention). Forward-only (compute ceiling) and end-to-end (including detection + COMPOSE) nearly coincide, so the reported win is not hidden in preprocessing. The gain rises with length, confirming that CTA operates orthogonally to FlashAttention.

(1.83× at 4096) is nearly identical to the forward-only figure (1.91×): the preprocessing cost is negligible (< 1% of wall-clock), not hidden. Reversibility on the 3B embedding space holds at 7.45×10^{-9}.

Length	m	m/n	Baseline (ms)	CTA (ms)	**Speedup**	+overhead
512	383	0.75	1167	898	**1.30**×	1.28×
1024	723	0.71	2575	1953	**1.32**×	1.30×
2048	1377	0.67	6693	3970	**1.69**×	1.67×
4096	2318	0.57	14215	7445	**1.91**×	1.83×

Table 9: Measured prefill wall-clock on Qwen2.5-3B, NVIDIA Tesla T4, FlashAttention (SDPA) backend, bf16, median of repeats. **Speedup** is forward-only; the final column adds span-detection and COMPOSE overhead. The win survives FlashAttention and grows with length because it rides on the length-linear cost (MLP/projections/head), which FA does not optimize.

Bonus: effective context extension. Because COMPOSE shortens the sequence losslessly, a context that overflows the model's positional window in full form can fit after collapse. On the same corpus, up to ∼1900 raw tokens collapse to $m \leq 1024$ and thus *fit* GPT-2's 1024-token window, where the baseline overflows outright—an effective context extension of ∼1.9×. This is a genuine, if corpus-dependent, side benefit: the extension factor equals the achieved collapse ratio and therefore, like the speedup, scales with the interchangeability of the context's repeats. It is not a replacement for architectural long-context methods, and it degrades on shallow-redundancy inputs where little collapse is possible.

9 Discussion, Limitations, and Conclusion

CTA reframes repeated spans as amortizable units of computation. Its defining property is *reversibility*: because COMPOSE is losslessly invertible, the model can operate on a compact opaque sequence by default and pay for full resolution only where a learned, compute-aware gate deems it necessary. Empirically, this yields a favorable frontier on deep-redundancy data (code: $\sim$50% attention FLOPs at improved perplexity) and a learned gate that strictly dominates a strong heuristic (up to 1.23 nats).

Applicability boundary. The central, honest finding is that CTA's benefit is *content-dependent*. On *deep* redundancy—code, boilerplate, duplicated prompts where repeated spans are truly interchangeable—collapse is nearly free and the gate expands little. On *shallow* redundancy—structured logs and tickets where a repeated template (`"user_id":`) immediately precedes distinguishing content (the value)—pooling destroys exactly the predictive signal, and the gate must expand aggressively, eroding the savings. CTA is therefore best understood not as a universal compressor but as a mechanism whose gain scales with the *interchangeability* of a context's repeats.

Limitations. (i) The gate is trained on *marginal* per-span utility rather than end-to-end through the full sequence, which is cheaper but less exact. (ii) Wall-clock is measured for the *prefill* pass—on CPU (Sec. 8) and on GPU with FlashAttention (Sec. 8.2, up to 1.91×); decode-time throughput with variable-length batching and paged-KV-cache reuse remains to be measured. (iii) The main qualitative experiments (reversibility, gated frontier, learned gate) use a small frozen GPT-2 and modest, partly synthetic corpora. Wall-clock is additionally validated on Qwen2.5-0.5B (CPU) and on *GPU with FlashAttention* (Qwen2.5-3B, Tesla T4, SDPA/FA: prefill speedup $1.30\times \to 1.91\times$ over $512 \to 4096$ tokens), where the mechanism ports without change and both reversibility and the length-driven speedup trend are confirmed; end-to-end decode with a paged KV-cache, and validating the qualitative results on larger modern models and held-out real corpora, are future work. (iv) The gate's query feature is a consumer-token embedding; the specification's $\gamma(q, \bar{e})$ with q a hidden state is strictly more expressive and requires intermediate activations.

Future work. End-to-end gate training; per-layer CTA (expand in lower layers, stay collapsed in upper layers, a dual-clock attention); composition with byte/patch tokenizers (BLT, H-Net) treating patches as CTA atoms; and near-duplicate (semantic, not just lexical) span classes via content-addressable attention.

Conclusion. To our knowledge CTA is the first reversible, session-scoped, parameter-free deduplication mechanism motivated by compute amortization. Reversibility turns the classic lossy-merge dilemma into a per-query granularity choice, and a small compute-aware gate learns to exploit it. The approach is a clear win where repeats are interchangeable, and its limits are precisely characterized where they are not.

References

[1] A. Pagnoni et al. *Byte Latent Transformer: Patches Scale Better Than Tokens.* arXiv:2412.09871, 2024.

[2] S. Hwang, B. Wang, A. Gu. *Dynamic Chunking for End-to-End Hierarchical Sequence Modeling (H-Net).* arXiv:2507.07955, 2025.

[3] *Breaking Token Into Concepts: Aggregate Semantic Grouping for Vocabulary Compression.* Findings of EMNLP, 2025.

[4] *R3Mem: Bridging Memory Retention and Retrieval via Reversible Compression.* arXiv:2502.15957, 2025.

[5] P. Nawrot et al. *Efficient Transformers with Dynamic Token Pooling.* ACL, 2023. arXiv:2211.09761.

[6] D. Bolya et al. *Token Merging: Your ViT But Faster.* ICLR, 2023. arXiv:2210.09461.

[7] F. Gloeckle et al. *Better & Faster Large Language Models via Multi-token Prediction.* ICML, 2024.